

UNIT - I

SYLLABUS

Fundamentals : Introduction, Models and Features, Concept of Distributed Operating system, Issues in Design of a Distributed Operating System.
 Foundations of Distributed System: Limitations of Distributed Systems, Lamport's logical clocks, Vector clocks, Causal ordering of messages, Global state recording, Cuts of a Distributed Computation, Termination Detection.

INTRODUCTION

A distributed system is a collection of independent computers that appear to the users of a system as a single computer. The main purpose of this chapter is to serve as an introduction to distributed operating system. This chapter presents the issues that arise in the design of a distributed operating system. In addition, we discuss the inherent limitations of distributed systems caused by the lack of common memory and a system wide common clock that can be shared by all the processes. The rest of the chapter is devoted to the discussion of how to overcome these inherent limitations. The theoretical foundations are the most fundamental to distributed computing.

FUNDAMENTALS OF DISTRIBUTED SYSTEMS

Q.1. What do you mean by distributed system?

Ans. Distributed system :

- A distributed system is a collection of processors. These processors are interconnected by a communication network. In this network each processor will have its own local memory. It will also have its own peripherals.
- Since the processors in the distributed system may be located far from each other, the communication between them takes place by message passing over the communication network. For each processor its own resources are local and other resources are remote.
- The distributed system gives a single system view. For this feature, a middleware service (software) is the necessity. This middleware service is placed between the higher level layer of users and applications and a lower level consisting of operating system.

This concept can be visualized as follow :

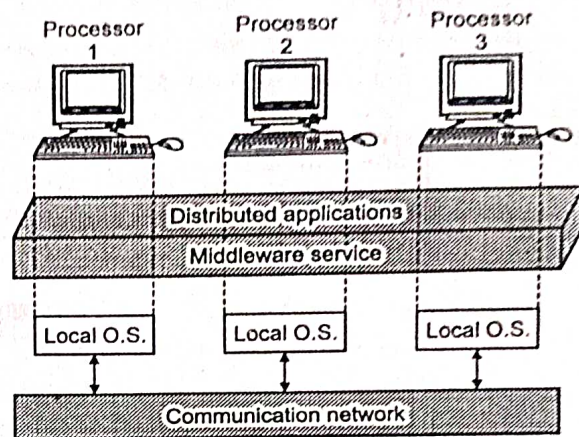


Fig. Distributed system as middleware

- The sharing of resources is a main motivation for constructing distributed system. Resources may be managed by servers and accessed by clients or they may be encapsulated as objects and accessed by other client objects.
- The distributed system has the following significant consequences :
 - Concurrent execution :**
 - Since there are n computers in the distributed environment each sharing the resources, all resources are available to all users without the feeling of interaction.
 - The capacity of the network can be increased by adding more resources to the network. Thus concurrent program execution is an important feature of the distributed system.
 - Independent failures :**
 - A computer may lead to a failure. So also the computer in a network may lead to failure. If a computer in a distributed system fails, their failure is not immediately made known to other components in the system. Fault in the computer isolates it from the system, but the system does not stop running.

- Faults in the network, are not made known to other components of the network. Each component of the network can fail independently, still leaving others running successfully.

FEATURES

Q.2. What are the characteristics/features of distributed system?

Ans. The main characteristics of distributed system are as follows :

(1) **Multiple autonomous components (computers):**

In a distributed system there are multiple components or computer. Each computer is an individual autonomous node having its own software and resources. It can also share components and resources from the distributed environment.

(2) **Components not shared by all users :**

Since a distributed system is a collection of autonomous computers linked by a computer network that appear to the users of the system at a single user, the user might not share some of the private resources in the distributed environment. So also, some of the resources may not be accessible by all the users.

(3) **Software architecture :**

- In a distributed environment the software runs in concurrent processes on different processors. The software architecture and the operating system must be such that it necessarily supports this feature. Therefore the distributed system must possibly have heterogeneous machines and networks or clusters of workstations or servers.
- Modern operating system like UNIX or NT allows an easy construction of a basic clusters of workstations.

(4) **Multiple management points :**

- The important characteristics of a distributed system is that there are multiple points of control.
- For example, automatic banking. In a distributed environment of automatic banking the primary requirements are security and reliability. Since there are multiple points of control, it is a challenge to maintain consistency of replicated data.
- Also, concurrent transactions which have operation involving accounts in different banks is an important feature.

- Since there is simultaneous access from several users, there may be multiple points of failure. So again fault tolerance is an important characteristic of any distributed system.

Q.3. What are the advantages of distributed operating system?

Ans. The advantages of distributed operating system are as follows :

- (1) **Resource Sharing :** Due to communication between connected computers resources can be shared among computers.
- (2) **Enhance Performance :** This is due to the fact that many tasks can be executed concurrently at different computers. Load distribution among computers can further improve response time.
- (3) **Improved reliability and availability :** Increased reliability is due to the fact that if few computers fail others are available and hence the system continues.
- (4) **Modular expandability :** New hardware and software resources can be added without replacing the existing resources.

The disadvantages of distributed operating system are as follows :

- (1) The lack of common memory and system wide common clock is an inherent problem in distributed systems.
- (2) Without a shared memory, up-to-date information about the state of the system is not available to every process via a simple memory lookup. The state information must therefore be collected through communication.
- (3) In the absence of global time, it becomes difficult to talk about temporal order of events. The combination of unpredictable communication delays and the lack of global time in a distributed system make it difficult to know how up-to-date collected state information really is.

MODELS

Q.4. Explain distributed operating system architecture types.

OR Write short note on distributed computing models.

Ans. Distributed computing models :

- Distributed systems can be modeled into several types. Various models are used for building distributed computing systems.
- These models can be broadly classified into five categories, and they are described below :

(1) Mini Computer Model.

(2) Workstation Model.

(3) Workstation Server Model.

(4) Processor Pool Model.

(5) Hybrid Model.

(7) Mini computer model :

In this model, the distributed system consists of several minicomputers. Each computer supports multiple users and provides access to remote resources. The ratio of processors to users is normally less than one.

Minicomputer model is a simple extension of the centralized time-sharing system. As shown in Fig.(a), a distributed computing system based on this model consists of a few minicomputers.

They may be large supercomputers as well interconnected by a communication network. Each minicomputer usually has multiple users simultaneously logged on to it.

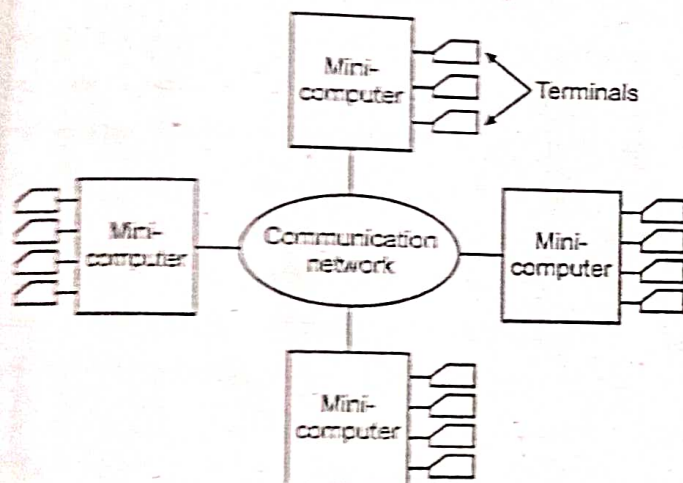


Fig. (a) The distributed system based on minicomputer model

Several interactive terminals are connected to each minicomputer. Each user is logged on to one specific minicomputer, with remote access to other minicomputers.

The network allows a user to access/remote resources that are available on some machine other than the one on to which the user is currently logged.

The minicomputer model may be used when resource sharing (such as sharing of information databases of different types, with each type

of database located on a different machine) with remote users is desired.

- The early ARPANET is an example of a distributed computing system based on the minicomputer model.

(2) Workstation model :

- In this model, the distributed system consists of several workstations; every user has a work-station where user's work is performed.
- With the help of distributed file system, a user can access data regardless of the location of the data. The ratio of processors to users is normally one.
- The workstations are independent computers with memory, hard disks, keyboard and console. Workstations are connected with each other through communication network.
- As shown in Fig.(b), a distributed computing system based on the workstation model consists of several workstations interconnected by a communication network.
- A company's office or a university department may have several workstations scattered throughout a building or campus each workstation equipped with its own disk and serving as a single-user computer.
- It has been often found that in such an environment at any one time (especially at night), a significant proportion of the workstations are idle, resulting in the waste of large amounts of CPU time.
- Therefore, the idea of the workstation model is to interconnect all these workstations by a high-speed LAN so that idle workstations may be used to process jobs of users who are logged onto other workstations and do not have sufficient processing power at their own workstations to get their jobs processed efficiently.
- In this model a user logs onto one of the workstations called his or her home workstation and submits jobs for execution.
- When the system finds that the user's workstation does not have sufficient processing power for executing the processes of the submitted jobs efficiently, it transfers one or more of the processes from the user's workstation to some other workstation that is currently idle and gets the process executed there, and finally the result of execution is returned to the user's, in workstation.

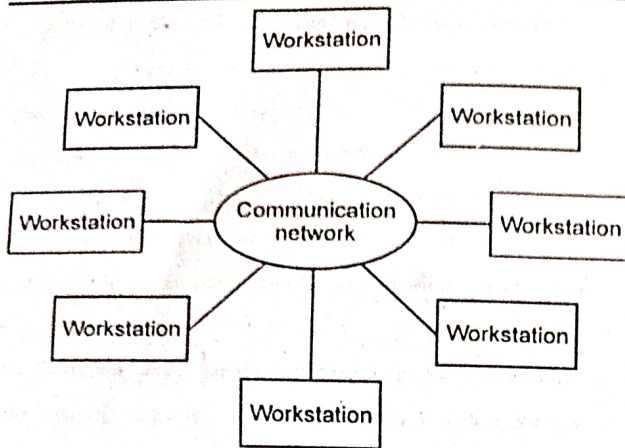


Fig. (b) A distributed system based on the workstation model

- This model is not so simple to implement because several issues must be resolved.
- Three commonly used approaches for handling the issues are as follows:
 - (i) The first approach is to allow the remote process share the resources of the workstation along with its own logged on user's processes. This method is easy to implement, but it defeats the main idea of workstations serving as personal computers, because if remote processes are allowed to execute simultaneously with the logged-on user's own processes, the logged-on user does not get his or her guaranteed response.
 - (ii) The second approach is to kill the remote process. The main drawbacks of this method are that all processing done by the remote process gets lost and the file system may be left in an inconsistent state; making this method unattractive.
 - (iii) The third approach is, migrate the remote process back to its home workstation, so that its execution can be continued there.
- This method is difficult to implement because it requires the system to support preemptive process migration facility.
- The Sprite system developed at Xerox is an examples of distributed computing systems based on the workstation model.
- (3) **Workstation-Server model :**
 - A workstation with its own local disk is usually called a diskfull workstation and a workstation without a local disk is called a diskless workstation.

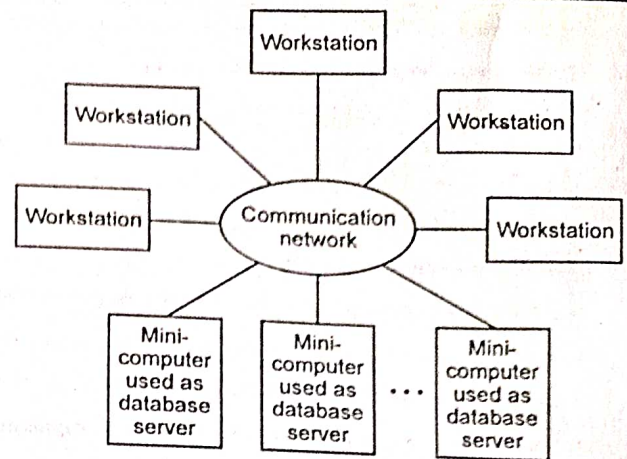


Fig. (c) A distributed system based on the workstation-server model

- With high-speed networks, diskless workstations have become more popular than diskfull workstations, making the workstation-server model more popular than the workstation model for building distributed computing systems.
- As shown in Fig. (c), a distributed computing system based on the workstation-server model consists of a few minicomputers and several workstations interconnected by a communication network. Most of the workstation may be diskless, but a few of may be disk full.
- When diskless workstations are used on a network, the file system to be used by these workstations must be implemented either by a disk full workstation or by a minicomputer equipped with a disk for file storage.
- One or more of the minicomputers are used for implementing the file system. Other mini-computers may be used for providing other types of services, such as database service and print service. Therefore, each minicomputer is used as a server machine to provide one or more types of services.
- For a number of reasons, such as higher reliability, and better scalability, multiple servers are often used for managing the resources of a particular type in a distributed computing system.
- For example, there may be multiple file servers, each running on a separate minicomputer and cooperating via the networks for managing the files of all the users in file system.
- In this model, a user logs onto a workstation called his or her home workstation.

- Normal computation activities required by the user's processes are performed at the user's home workstation, but requests for services provided by special servers (such as a file server or a database server) are sent to a server providing that type of service that performs the user's requested activity and returns the result of request processing to the user's workstation. Therefore, in this model, the user's processes need not be migrated to the server machine for getting the work done by those machines.
- For better overall system performance the local disk of a diskfull workstation is normally used for such purposes as storage of temporary files, storage of unshared files, storage of shared files that are rarely changed, paging activity in virtual-memory management, and caching of remotely accessed data.
- As compared to the workstation model, the work-station-server model has several advantages :
 - (i) It is much cheaper to use a few minicomputers equipped with large fast disks that are accessed over the network than a large number of diskfull workstations, with each workstation having a small, slow disk.
 - (ii) Diskless workstations are also preferred to diskfull workstations from a system maintenance point of view. Software installation, backup and hardware maintenance are easier to perform with a few large disks than with many small disks scattered all over a building or campus.
 - (iii) In the workstation-server model, since all files are managed by file servers, users have the flexibility to use any workstation and access the files in the same manner irrespective of which workstation the user is currently logged on. Note that this is not true with the workstation model, in which each workstation has its local file system, because different mechanisms are needed to access local and remote files.
 - (iv) In the workstation-server model, the request-response protocol described above is mainly used to access the services of the server machine. Therefore, unlike the workstation model, this model does not need a process migration facility which is difficult to implement.
- The request-response protocol is known as the client-server model of communication. In this model, a client process (which in this case resides on a workstation) sends a request to a server process (which in this case resides on a computer) for getting some service such as

reading a block of a file to the server executes the request and sends back a reply to the client that contains the result of request processing.

- The client-server model provides an effective general-purpose approach to the sharing of information and resources in distributed computing systems. It is not only meant for use with the workstation-server model but also can be implemented in a variety of hardware and software environments. The computers used to run the client and server processes need not necessarily be workstations and minicomputers.
 - They can be of many types and there is no need to distinguish between them. It is even possible for both the client and server processes to be run on the same computer. Moreover some processes are both client and server processes. That is, a server process may use the services of another server, appearing as a client to the latter.
 - A user has guaranteed response time because workstations are not used for executing remote processes. However the model does not utilize the processing capability of idle workstations.
- (4) **Processor pooled model :**
- Processor-pool model is based on the observation that most of the time a user does not need any computing power but once in a while he or she may need a very large amount of computing power for a short time. Therefore, in the processor-pooled model the processors are pooled together to be shared by the users as needed.

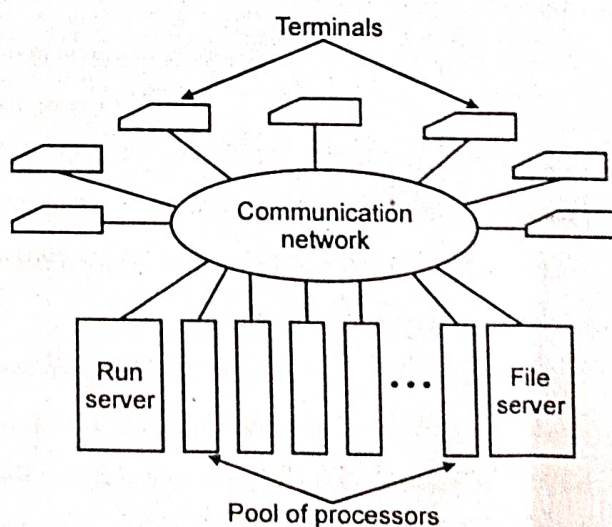


Fig. (d) A distributed computing system based on processor-pool model

- The pool of processors consists of a large number of microcomputer and minicomputers attached to the network.
- Each processor in the pool has its own memory to load and run a system program or an application program of the distributed computing system.
- In the pure processor's model, the processors in the pool have no terminals attached directly to them, and users access the system from terminals that are attached to the network via special devices. These terminals are either small diskless workstations or graphic terminals.
- A special server, called a run server, manages and allocates the processors in the pool to different users on a demand bases. When a user submits a job for computation, an appropriate number of processors are temporarily assigned to the job by the run server.
- As compared to the workstation-server model, the processor-pool model allows better utilization of the available processing power of a distributed computing system.
- In the processor-pool model, the entire processing power of the system is available for use by the currently logged-users, whereas this is not true for the workstation-server model in which several workstations may be idle at a particular time but they cannot be used for processing the jobs of other users.
- Furthermore, the processor-pool model provides greater flexibility than the workstation-server model in the sense that the system's services can be easily expanded without the need to install any more computers.
- The professors in the pool can be allocated to act as extra servers to carry any additional load arising from an increased user population or to provide new services.
- However, the processor-pool model is usually considered to be unsuitable for high-performance interactive applications, especially those using graphics or window systems.
- This is mainly because of the slow speed of communication between the computer on which the application program of a user is being executed and the terminal via which the user is interacting with to system. The workstation-server model is genially considered to be more suitable for such applications.

- Out of the four models describe above, the workstation-server model is the most widely used model for building distributed computing systems. This is because a large number of computer users only perform simple interactive tasks such as editing jobs, sending electronic mails, and executing small programs.
- The workstation-server model is ideal for such simple usage. However, in a working environment that has groups of users who often perform jobs going massive computation the processor-pool model is more attractive and suitable.
- In the processor pool model, the ratio of processors to users is greater than one. The model attempts to allocate one or more processors to a user to complete the task.
- Once the user's task is completed the assigned processors are returned to the pool.

(5) Hybrid model :

- To combine the advantages of both the workstation-server and processor-pool models, a hybrid model may be used to build a distributed computing system. The hybrid model is based on the workstation-server model but with the addition of a pool of processors. The processors in the pool can be allocated dynamically for computations that are too large for workstations or that requires several computers concurrency for efficient execution.
- In addition to efficient execution of computation-intensive jobs, the hybrid model gives guaranteed response to interactive jobs by allowing them to be processed on local workstations of the users. However, the hybrid model is more expensive to implement than the workstation-server model or the processor-pool model.

CONCEPT OF DISTRIBUTED OPERATING SYSTEM

Q.5. Explain the concept of distributed operating system.

Ans. Distributed operating system :

- An operating system is a program that manages the resources of computer systems and provides users with a friendly interface to the system.
- A distributed operating system extends the concepts of resource management and user friendly interface for the shared memory computers a step further, encompassing as distributed computing system consisting of several autonomous computers connected by a communication network.

- A distributed operating system appears to its users as a centralized operating system for a single machine, but it runs on multiple-independent computers.
- An identical copy of the operating system (or a different operating system providing similar services) may run at every computer.
- On the other hand, some computers in the system that serve a special purpose might run an extended version of the operating system.
- The key concept is transparency.

- In other words, the use of multiple processors and the accessing of remote data should be invisible (transparent) to the user.
- The user views the system as a virtual uniprocessor, and not as a collection of distinct machines.
- For instance, a user simply submits a job to the distributed operating system through a computer. The user does not know on what computers the job was executed, on what computers the files needed for execution were stored, or how the communication and synchronization among different computers were carried out.

Q.6. Draw a neat and labeled diagram and explain architecture of a distributed system.

Ans. Architecture of a distributed system :

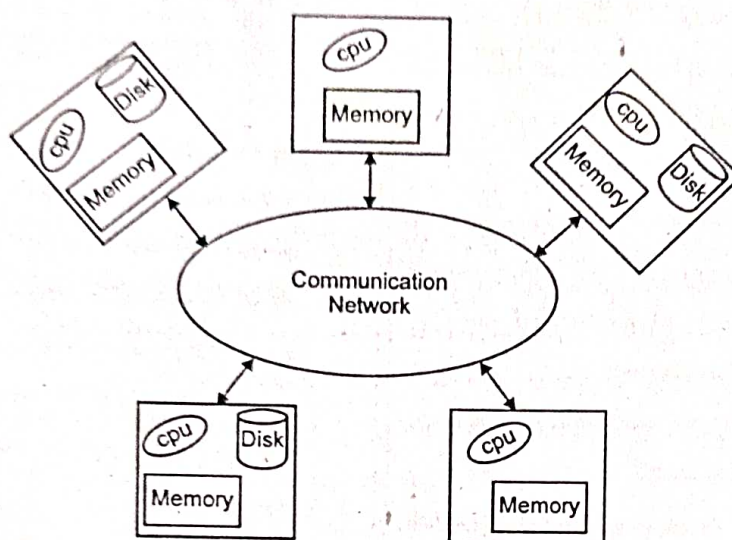


Fig. Architecture of distributed system

- The distributed operating system consists of several computers that do not share a memory or a clock.
- The computers communicate with each other by exchanging messages over a communication network.
- Each computer has its own memory and runs its own operating system.
- The resources owned and controlled by other computers are said to be local to it.
- While the resources owned and controlled by other computers and those that can only be accessed through the network are said to be remote.
- Typically, accessing remote resources is more expensive than accessing local resources because of the communication delays that occur in the network and the CPU overhead incurred to process communication protocols.

Q.7. Explain software concepts of distributed system.

Ans. Software concepts :

- Although the hardware is important, the software is even more important. The image that a system presents to its users, and how they think about the system, is largely determined by the operating system software, not the hardware.

- By nature software is vague and amorphous. Still, it is more-or-less possible to distinguish two kind of operating systems for multiple CPU systems : loosely coupled and tightly coupled. As loosely and tightly-coupled software is roughly analogous to loosely and tightly-coupled hardware.

- Loosely-coupled software allows machines and users of a distributed system to be fundamentally independent of one another, but still to interact to a limited degree where that is necessary.
- Consider a group of personal computers, each of which has its own CPU, its own memory, its own hard disk, and its own operating system, but which share some resources, such as laser printers and data bases, over a LAN.
- This system is loosely coupled, since the individual machines are clearly distinguishable, each with its own job to do. If the network should go down for some reason, the individual machines can still continue to run to a considerable degree, although some functionality may be lost (e.g., the ability to print files).
- To show how difficult it is to make definitions in this area, now consider the same system as above, but without the network. To print a file, the user writes the file on a floppy disk carries it to the machine with the printer, reads it and then print it.
- From a fundamental point of view, there is not really any theoretical difference between communicating over a LAN and communicating by carrying floppy disks around. At most one can say that the delay and data rate are worse in the second example.
- At the other extreme we might find a multiprocessor dedicated to running a single chess program in parallel. Each CPU is assigned a board to evaluate, and it spend its time examining that board and all the boards that can be generated from it.
- When the evaluation is finished, the CPU reports back the results and is given a new board to work on. The software for this system, both the application program and the operating system required to support it, is clearly much more tightly coupled.
- In theory, there should be eight combinations of hardware and software. In fact, only four are worth distinguishing, because to the user, the interconnection technology is not visible.

Q.8. Explain network operating system in detail.

Ans. Network operating systems :

- Consider loosely-coupled software on loosely-coupled hardware, since this is probably the most common combination at many

organizations. A typical example is network of workstations connected by a LAN.

- In this model, each user has a workstation for his exclusive use. It may or may not have a hard disk. It definitely has its own operating system. All commands are normally run locally, right on the workstation.
- However, it is sometimes possible for a user to log into another workstation remotely by using a command such as : `rlogin machine`
- The effect of this command is to turn the user's own workstation into a remote terminal logged into the remote machine. Commands typed on the keyboard are sent to the remote machine, and output from the remote machine is displayed on the screen.
- To switch to a different remote machine, it is necessary first to log out, then to use the `rlogin` command to connect to another machine. At any instant, only one machine can be used, and the selection of the machine is entirely manual.
- Network of workstations often also have a remote copy command to copy files from one machine to another. For example, a command like, `rcp machine1:file1 machine2:file2` might copy the file `file1` and `machine1` to `machine2` and give it the name `file2` there.
- Again here, the movement of files is explicit and requires the user to be completely aware of where all files are located and where all commands are being executed.
- While better than nothing, this form the communication is extremely primitive and has led system designers to search for more convenient forms of communication and information sharing.
- One approach is to provide a shared, global file system accessible from all the workstations. The file system is supported by one or more machine called file servers.
- The file servers accept requests from user programs running on the other (nonserver) machines, called clients to read and write files. Each incoming request is examined and executed and the reply is sent back, as illustrated in Fig.(a).

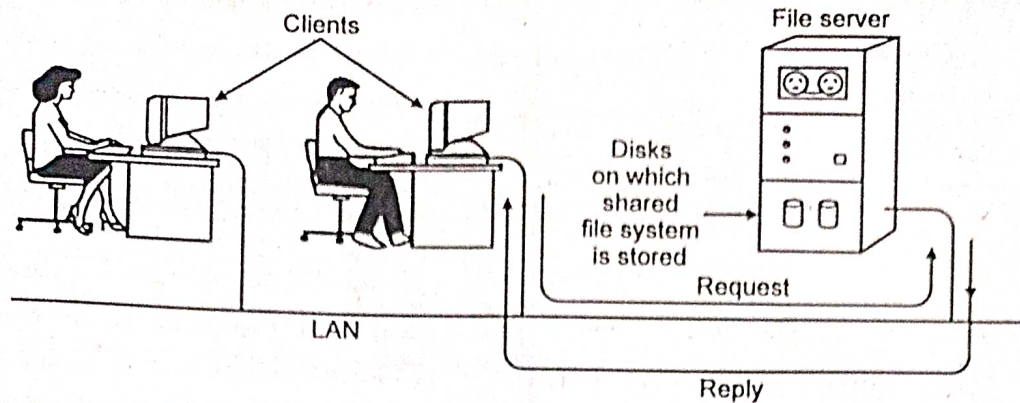
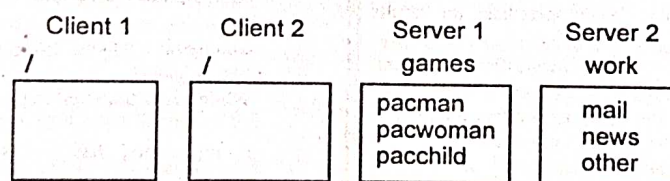
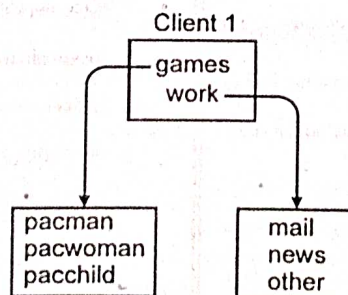


Fig. (a) Two clients and a server in a network operating system

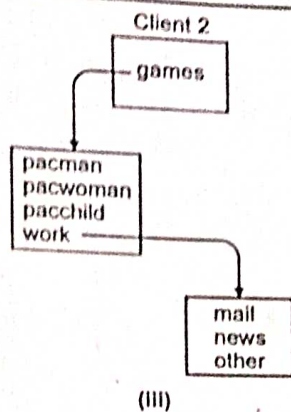
- File servers generally maintain hierarchical file system, each with a root directory containing subdirectories and files. Workstations can import or mount these file system, augmenting their local file systems with those located on the servers.
- For example, in Fig. (b) two file servers are shown. One has a directory called games, while the other has a directory called work. These directories each contain several files.
- Both of the clients shown have mounted both of the servers, but they have mounted them in different places in their respective file system. Client1 has mounted them in its root directory, and can access them as /games and /work, respectively.
- Client2, like client1, has mounted games in its root directory, but regarding the reading of mail and news as a kind of game, has created a directory /games/work and mounted work there. Consequently, it can access news using the path / games / work/ news rather than / work/ news.
- While it does not matter where a client mounts a server in its directory hierarchy, it is important to notice that different clients can have a different view of the file system.
- The name of a file depends on where it is being accessed from, and how that machine has set up its file system. Because each workstation operates relatively independently of the others, there is no guarantee that they all present the same directory hierarchy to their programs.
- The operating system that is used in this kind of environment must manage the individual workstations and file servers and take care of the communication between them.



(i)



(ii)



(III)

Fig. (b) Different clients may mount the servers in different places

- It is possible that the machines all run the same operating system, but this is not required. If the clients and servers run an different systems, as a bare minimum they must agree on the format and meaning of all the messages that they may potentially exchange.
- In a situation like this, where each machine has a high degree of autonomy and there are few system-wide requirements, people usually speak of a network operating system.

Q.9. Explain true distributed system in detail.

Ans. True distributed system :

- Network operating systems are loosely coupled software on loosely-coupled hardware. Other than the shared file system, it is quite apparent to the users that such a system consists of numerous computers.
- Each can run its own operating system and do whatever its owner wants. There is essentially no coordination at all, except for the rule that client-server traffic must obey the system's protocols.
- The next evolutionary step beyond this is tightly-coupled software on the same loosely-coupled (i.e., multicomputer) hardware.
- The goal of such a system is to create the illusion in the minds of the users that the entire network of computers is a single-timesharing system, rather than a collection of distinct machines. Some author refer to this property as the single-system image.
- Others put it slightly differently, saying that a distributed system is one that runs on a collection of networked machines but acts like a virtual uniprocessor.

- No matter how it is expressed, the essential idea is that the users should not have to be aware of the existence of multiple CPUs in the system. No current system fulfills this requirement entirely, but a number of candidates are on the horizon.
- It will not do to have different mechanism on different machines or different mechanisms for local communication and remote communication. There must also be a global protection scheme. Mixing access control lists, the UNIX protection bits, and capabilities will not give a single system image.
- Process management must also be the same everywhere. How processes are created, destroyed, started, and stopped must not vary from machine to machine. In short, the idea behind network operating system, namely that any machine can do whatever it wants to as long as it obeys the standard protocols when engaging in client server communication, is not enough.
- Not only there be a single set of system calls available on all machines, but these calls must be designed so that they make sense in a distributed environment.
- The file system must look the same everywhere, too. Having file names restricted to 11 characters in some locations and being unrestricted in others is undesirable. Also every file should be visible at every location, subject to protection and security constraints, of course.
- As a logical consequence of having the same system call interface everywhere. It is normal that identical kernels run on all the CPUs in the system. Doing so makes it easier to coordinate activities that must be global.
- For example, when a process has to be started up, all the kernels have to cooperate in finding the best place to execute it. In addition, a global file system needed.
- Nevertheless, each kernel can have considerable control over its own local resources. For example, since there is no shared memory, it is logical to allow each kernel on each CPU is the logical place to determine what to swap or page. There is no reason to centralize this authority. Similarly, if multiple processes are running on some CPU, it makes sense to do the scheduling right there, too.
- A considerable body of knowledge is now available about designing and implementing distributed operating systems.

Q.10. Explain multiprocessor timesharing system.

Ans. Multiprocessor timesharing system :

- The most common general purpose examples are multiprocessor that are operated as a UNIX timesharing system but with multiple CPUs instead of one CPU.
- To the outside world, a multiprocessor with 32-30 MIPS CPUs acts very much like a single 960-MIPS CPU.
- Except that implementing it on a multiprocessor makes life much easier, since the entire design can be centralized.
- The key characteristic of this class of system is the existence of a single run queue, a list of all the processes in the system that are logically unblocked and ready to run. The run queue is a data structure kept in the shared memory.
- As an example, consider the system of Fig. which has three CPUs and five processes that are ready to run. All five processes are located in the shared memory, and three of them are currently executing, process A on CPU1, process B on CPU2, and process C on CPU3. The other two processes, D and E, are also in memory waiting their turn.

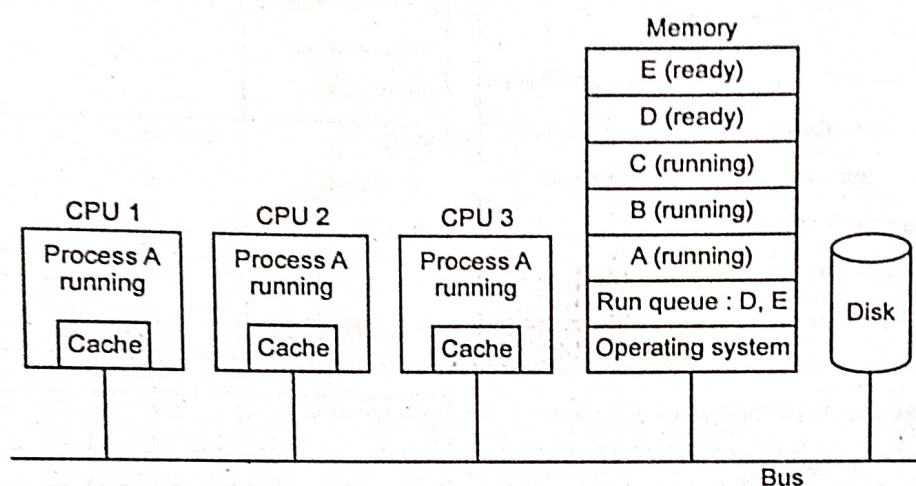


Fig. A multiprocessor with a single run queue.

- Now, suppose that process B blocks waiting for I/O or its quantum runs out. Either way, CPU2 must suspend it, and find another process to run CPU2 will normally begin executing operating system code (located in the shared memory).
- After having saved all of B's registers, it will enter a critical region to run the scheduler to look for another process to run. It is essential that the scheduler be run as a critical region to prevent two CPUs from choosing the same process to run next.
- The necessary mutual exclusion can be achieved by using monitors, semaphores, or any other standard construction used in singleprocessor systems.
- Once CPU2 has gained exclusive access to the run queue, it can remove the first entry, D, exit from the critical region, and begin executing D.
- Initially, execution will be slow, since CPU2's cache is full of words belonging to that part of the shared memory containing process B, but after a little while, these will have been purged and the cache will be full of D's code and data, so execution will speed up.
- Because none of the CPUs have local memory and all programs are stored in the global shared memory, it does not matter on which CPU a process runs. If a long-running process is scheduled many times before it completes, on the average, it will spend about the same amount of time running on each CPU.
- The only factor that has any effect at all on CPU choice is the slight gain in performance when a process starts up on a CPU that is currently caching part of its address space.

- In other words, if all CPUs are idle, waiting for I/O, and one process becomes ready, it is slightly preferable to allocate it to the CPU it was last using, assuming that no other process has used that CPU.
- As an aside, if a process blocks for I/O on a multiprocessor, the operating system has the choice of suspending it or just letting it do busy waiting. If most I/O is completed in less time than it takes to do a process switch, busy waiting is preferable.
- Some systems let the process keep its processor for a few multiseconds, in the hope that the I/O will complete soon, but if that does not occur before the timer runs out, a process switch is made.
- If most critical regions are short, this approach can avoid many expensive process switches.
- An area in which this kind of multiprocessor differs appreciably from a network or distributed system is in the organization of the file system. The operating system normally contains a traditional file system, including a single, unified block cache.
- When any process executes a system call, a trap is made to the operating system, which carries it out, using semaphores, monitors, or something equivalent, to lock out other CPUs while critical sections are being executed or central tables are being accessed.
- In this way, when a WRITE system call is done the central block cache is locked, the new data entered into the cache, and the lock released. Any subsequent READ call will see the new data, just as on a single-processor system.
- On the whole, the file system is hardly different from a single-processor file system. In fact, on some multiprocessors, one of the CPUs is dedicated to running the operating system, the other ones run user programs. This situation is undesirable, however, as the operating system machine is often a bottleneck.
- It should be clear that the methods used on the multiprocessor to achieve the appearance of a virtual uniprocessor are not applicable to machines that do not have shared memory.
- Centralized run queues and block only caches work when all CPUs have access to them with very low delay. Although these data structures could be simulated on a network of machines, the communication costs make this approach prohibitively expensive.
- Some of the differences between the three kinds of system is shown below:

Item	Network operating system	Distributed operating system	Multi-processor operating system
Does it look like a virtual uniprocessor?	No	Yes	Yes
Do all have to run the same operating system?	No	Yes	Yes
How many copies of the operating system are there?	N	N	1
How is communication achieved?	Shared files	Messages	Shared memory
Are agreed upon network protocols required?	Yes	Yes	No
Is there a single run queue?	No	No	Yes
Does file sharing have well-defined semantics?	Usually no	Yes	Yes

Q.11. Explain hardware concept of distributed system.

Ans. Hardware concepts :

- Even though all distributed systems consist of multiple CPUs, there are several different ways the hardware can be organized, especially in terms of how they are interconnected and how they communicate.
- Various classification schemes for multiple CPU computer systems have been proposed over the years, but none of them have really caught on and been widely adopted. Probably the most frequently cited taxonomy is Flynn's (1972), although it is fairly rudimentary. Flynn picked two characteristics that he considered essential: the number of instruction streams and the number of data streams.
- A computer with a Single Instruction Stream and a Single Data Stream is called SISD. All traditional uniprocessor computers (i.e., those having only one CPU) fall in this category, from personal computers to large mainframes.

The next category is SIMD, single instruction stream, multiple data stream. This type refers to array processors with one instruction unit that fetches an instruction, and then commands many data units to carry it out in parallel, each with its own data.

These machines are useful for computations that repeat the same calculation on many sets of data, for example, adding up all the elements of 64 independent vectors. Some supercomputers are SIMD.

The next category is MISD, multiple instruction stream, single data stream. No known computers fit this model. Finally, comes MIMD, which essentially means a group of independent computers, each with its own program counter, program, and data. All distributed systems are MIMD, so this classification system is not tremendously useful for our purposes.

In following Fig., we divide all MIMD computers into two groups those that have shared memory, usually called multiprocessors, and those that do not, sometimes called multicomputers. The essential difference is this: in a multiprocessor, there is a single virtual address space that is shared by all CPUs.

If any CPU writes, for example, the value 44 to address 1000, any other CPU subsequently reading from its address 1000 will get the value 44. All the machines share the same memory.

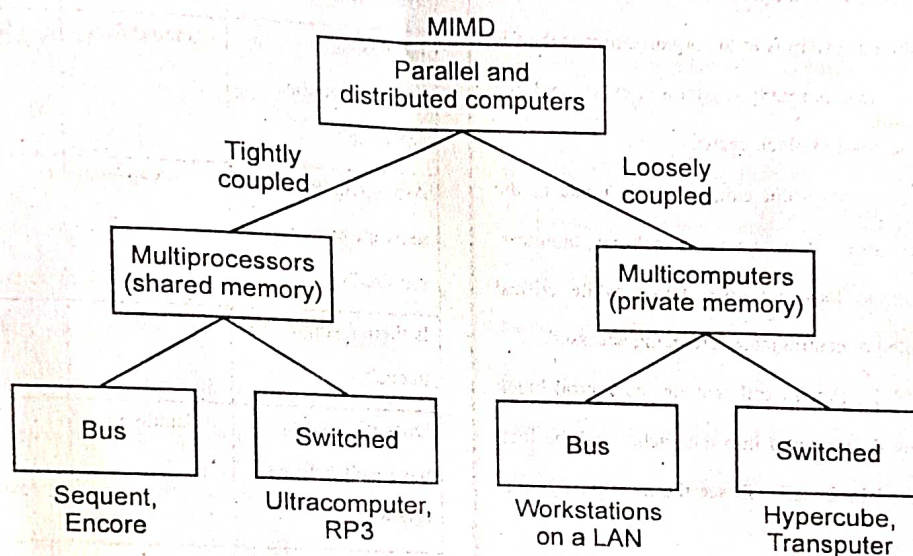


Fig. A taxonomy of parallel and distributed computer systems

In contrast, in a multicomputer, every machine has its own private memory. If one CPU writes the value 44 to address 1000, when another CPU reads address 1000 it will get whatever value was there before. The write of 44 does not affect its memory at all. A common example of a multicomputer is a collection of personal computers connected by a network.

Each of these categories can be further divided based on the architecture of the interconnection network. In Fig. we describe these two categories as bus and switched. By bus we mean that there is a single network, backplane, bus, cable, or other medium that connects all the machines. Cable television uses a scheme like this: the cable company runs a wire down the street, and all the subscribers have taps running to it from their television sets.

Q.12. Explain bus based multiprocessors.

Ans. Bus based multiprocessors :

- Bus based multiprocessor consist of some number of CPUs all connected to a common bus, along with a memory module.

- A simple configuration is a high-speed backplane or motherboard into which CPU and memory cards can be inserted.
- A typical bus has 32 or 64 address lines, 32 or 64 data lines, and perhaps 32 or more control lines, all of which operate in parallel.

- To read a word memory, a CPU puts the address of the word it wants on the bus address lines then puts a signal on the appropriate control lines to indicate that it wants to read.
- The memory responds by putting the value of the word on the data lines to allow the requesting CPU to read it in. Writes work in a similar way.
- Since there is only one memory, if CPU A writes a word to memory and then CPU B reads that word back a microsecond later, B will get the value just written. A memory that has this property is said to be coherent.
- Coherence plays an important role in distributed operating systems in a variety of ways.
- The problem with this scheme is that with as few as 4 or 5 CPUs, the bus will usually be overloaded and performance will drop drastically.
- The solution is to add a high-speed cache memory between the CPU and the bus, as shown in Fig.
- The cache holds the most recently accessed words. All memory requests go through the cache. If the words requested is in the cache, the cache itself responds to the CPU, and no bus request is made.
- If the cache is large enough, the probability of success, called the hit rate, will be high, and the amount of bus traffic per CPU will drop dramatically, allowing many more CPUs in the system. Cache sizes of 64 K to 1M are common, which often gives a hit rate of 90 percent or more.

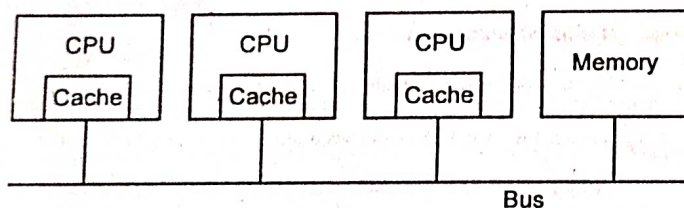


Fig. A bus-based multiprocessor

- However, the introduction of cache also brings a serious problem with it. Suppose that two CPUs, A and B, each read the same word into their respective caches. Then A overwrites the word. When B next reads that word, it gets the old value from its cache, not the value A just wrote. The memory is now incoherent, and the system is difficult to program.

- Suppose that the cache memories are designed so that whenever a word is written to the cache, it is written through to memory as well. Such a cache is, not surprisingly, called a write-through cache.
- In this design, cache hits for reads do not cause bus traffic, but cache misses for reads, and all writes, hits and misses, cause bus traffic.
- In addition all caches constantly monitor the bus. Whenever a cache sees a write occurring to a memory address present in its cache, it either removes that entry from its cache, or updates the cache entry with the new value. Such a cache is called a snoopy cache (or sometimes, a snooping cache) because it is always "snooping" (eaves-dropping) on the bus. A design consisting of snoopy write through caches is coherent and is invisible to the programmer. Nearly all bus-based multiprocessors use either this architecture or one closely related to it.

Q.13. Explain switched multiprocessors.

Ans. Switched multiprocessors :

- To build a multiprocessor with more than 64 processors, a different method is needed to connect the CPUs with the memory. One possibility is to divide the memory up into modules and connect them to the CPUs with a crossbar switch, as shown in Fig. (a).
- Each CPU and each memory has a connection coming out of it, as shown in fig.(a). At every intersection is a tiny electronic crosspoint switch that can be opened and closed in hardware.
- When a CPU wants to access a particular memory, the crosspoint switch connecting them is closed momentarily, to allow the access to take place.
- The virtue of the crossbar switch is that many CPUs can be accessing memory at the same time, although if two CPUs try to access the same memory simultaneously, one of them will have to wait.
- The downside of the crossbar switch is that with n CPUs and n memories, n crosspoint switches are needed. For large n , this number can be prohibitive. As a result, people have looked for, and found, alternative switching networks that require fewer switches.

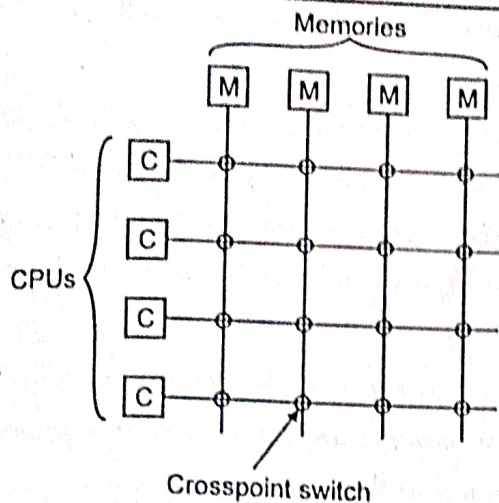


Fig. (a) A crossbar switch

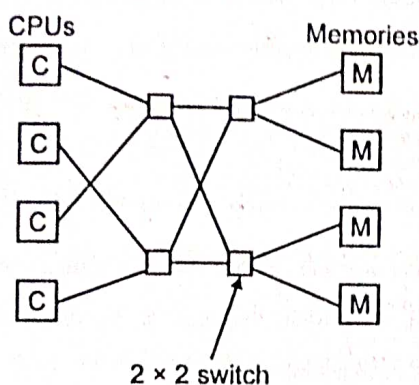


Fig. (b) An omega switching network

The omega network of Fig.(b) is one example. This network contains four 2×2 switches, each having two inputs and two outputs. Each switch can route either input to either output. A careful look at the figure will show that with proper settings of the switches, every CPU can access every memory. These switches can be set in nanoseconds or less.

In the general case, with n CPUs and n memories, the omega network requires $\log_2 n$ switching states, each containing $n/2$ switches, for a total of $(n \log_2 n)/2$ switches. Although for large n this is much better than n^2 , it is still substantial.

Furthermore, there is another problem delay. For example, for $n = 1024$, there are 10 switching stages from the CPU to the memory, and another 10 for the word requested to come back.

Suppose that the CPU is a modern RISC chip running at 100 MIPS; that is, the instruction execution time is 10 nsec. If a memory request is to traverse a total of 20 switching stages (10 outbound and 10

back) in 10 nsec, the switching time must be 500 picosec (0.5 nsec).

- The complete multiprocessor will need 5120 500-picosec switches. This is not going to be cheap.
- People have attempted to reduce the cost by going to hierarchical systems. Some memory is associated with each CPU. Each CPU can access its own local memory quickly, but accessing anybody else's memory is slower. This design gives rise to what is known as a NUMA (NonUniform Memory Access) machine.
- Although NUMA machines have better average access times than machines based on omega networks, they have the new complication that the placement of the program and data becomes critical in order to make most access go to the local memory.
- To summarize, bus-based multiprocessors, even with supply caches, are limited by the amount of bus capacity to about 64 CPUs at most. To go beyond that requires a switching network, such as a crossbar switch, an omega switching network, or something similar.
- Large crossbar switches are very expensive, and large omega networks are both expensive and slow. NUMA machines require complex algorithms for good software placement. The conclusion is clear building a large, tightly-coupled, shared memory multiprocessor is possible, but is difficult and expensive.

Q.14. Explain bus-based multicomputers.

Ans. Bus-based multicomputers :

- On the other hand, building a multicomputer (i.e. no shared memory) is easy. Each CPU has a direct connection to its own local memory. The only problem left is how the CPUs communicate with each other.
- Clearly, some interconnection scheme is needed here, too, but since it is only for CPU-to-CPU communication, the volume of traffic will be several orders of magnitude lower than when the interconnection network is also used for CPU to memory traffic.
- In Fig. we see a bus-based multicomputer. It looks topologically similar to the bus-based multiprocessor, but since there will be much less traffic over it, it need not be a high-speed backplane bus.

- In fact, it can be a much lower speed LAN (typically 10-100 Mbps, compared to 300 Mbps and up for a backplane bus).
- Thus Fig. is more often a collection of workstations on a LAN than a collection of CPU cards inserted into a fast bus (although the latter configuration is definitely a possible design).

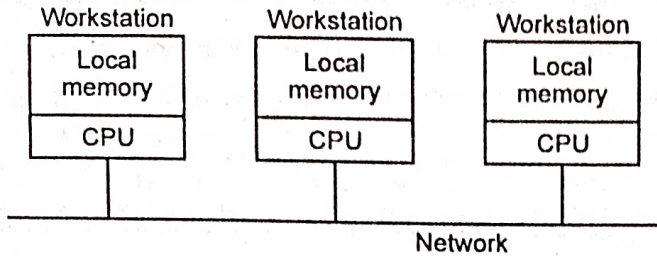


Fig. A multicomputer consisting of workstations on a LAN

Q.15. Explain switched multicomputers.

Ans. Switched multicomputers :

- Our last category consists of switched multi-computers. Various interconnection network have been proposed and built, but all have the property that each CPU has direct and exclusive access to its own, private memory.
- Fig. shows two popular topologies, a grid and a hypercube. Grids are easy to understand and lay out on printed circuit boards. They are best suited to problems that have an inherent two-dimensional nature, such as graph theory or vision (e.g. robot eyes or analyzing photographs).

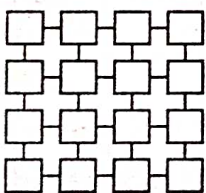
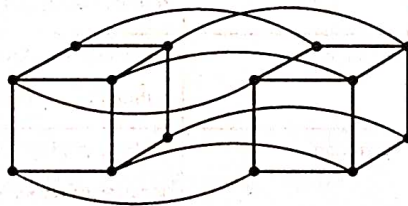


Fig. (a) Grid



(b) Hypercube

- A hypercube is an n -dimensional cube. The hypercube of Fig. (b) is four-dimensional. It can be thought of as two ordinary cubes, each with 8 vertices and 12 edges.
- Each vertex is a CPU. Each edge is a connection between two CPUs. The corresponding vertices in each of the two cubes are connected.
- To expand the hypercube to five dimensions, we would add another

set of two interconnected cubes to figure, connect the corresponding edges in the two halves, and so on.

- For an n -dimensional hypercube, each CPU has n connections to other CPUs. Thus, the complexity of the wiring increases only logarithmically with the size. Since only nearest neighbors are connected many messages have to make several hops to reach their destination.
- However, the longest possible path also grows logarithmically with the size, in contrast to the grid, where it grows as the square root of the number of CPUs.
- Hypercubes with 1024 CPUs have been commercially available for several years, and hypercubes with as many as 16,384 CPUs are starting to become available.

ISSUES IN DESIGN OF A DISTRIBUTED OPERATING SYSTEM

Q.16. Explain design issues of distributed operating system.

Ans. Some important issues that arise in the design of a distributed operating system are as follows :

- (1) Transparency.
 - (2) Reliability.
 - (3) Flexibility.
 - (4) Performance.
 - (5) Scalability.
 - (6) Heterogeneity.
 - (7) Security.
- (1) Transparency :
- Transparency or single-system image refers to the ability of an application to treat the system on which it operates without regard to whether it is distributed and without regard to hardware or other implementation details.
 - Many areas of a system can benefit from transparency, including access, location, performance, naming, and migration. The consideration of transparency directly affects decision making in every aspect of design of a distributed operating system. Transparency can impose certain requirements and/or restrictions on other design considerations.

- Systems can optionally violate transparency to varying degrees to meet specific application requirements. For example, a distributed operating system may present a hard drive on one computer as "C:" and a drive on another computer as "G:".
- The user does not require any knowledge of device drivers or the drive's location; both devices work the same way, from the application's perspective. A less transparent interface might require the application to know which computer hosts the drive.

Transparency domains:

(i) **Location transparency :**

- Location transparency comprises two distinct aspects of transparency, naming transparency and user mobility.
- Naming transparency requires that nothing in the physical or logical references to any system entity should expose any indication of the entity's location, or its local or remote relationship to the user or application.
- User mobility requires the consistent referencing of system entities, regardless of the system location from which the reference originates

(ii) **Access transparency :**

- Local and remote system entities must remain indistinguishable when viewed through the user interface.
- The distributed operating system maintains this perception through the exposure of a single access mechanism for a system entity, regardless of that entity being local or remote to the user.
- Transparency dictates that any differences in methods of accessing any particular system entity either local or remote must be both invisible to, and undetectable by the user.

(iii) **Migration transparency :**

Resources and activities migrate from one element to another controlled solely by the system and without user/application knowledge or action.

(iv) **Replication transparency :**

The process or fact that a resource has been duplicated on another element occurs under system control and without user/application knowledge or intervention.

- (v) **Concurrency transparency :** Users/applications are unaware of and unaffected by the presence/activities of other users.

- (vi) **Failure transparency :** The system is responsible for detection and remediation of system failures. No user knowledge/action is involved other than waiting for the system to resolve the problem.
- (vii) **Performance Transparency :** The system is responsible for the detection and remediation of local or global performance shortfalls. Note that system policies may prefer some users/user classes/tasks over others. No user knowledge or interaction is involved.
- (viii) **Size/Scale transparency :** The system is responsible for managing its geographic reach, number of nodes, level of node capability without any required user knowledge or interaction.
- (ix) **Revision transparency :** The system is responsible for upgrades and revisions and changes to system infrastructure without user knowledge or action.
- (x) **Control transparency :** The system is responsible for providing all system information, constants, properties, configuration settings, etc. in a consistent appearance, connotation, and denotation to all users and applications.
- (xi) **Data transparency :** The system is responsible for providing data to applications without user knowledge or action relating to where the system stores it.
- (xii) **Parallelism transparency :** The system is responsible for exploiting any ability to parallelize task execution without user knowledge or interaction. Arguably the most difficult aspect of transparency.

Transparency	Description
Access	Hide differences in data representation and resource access (enables interoperability).
Location	Hide location of resource (can use resource without knowing its location).
Migration	Hide possibility that a system may change location of resource (no effect on access).
Replication	Hide the possibility that multiple copies of the resource exist (for reliability and/or availability).
Concurrency	Hide the possibility that the resource may be shared concurrently.
Failure	Hide failure and recovery of the resource. How does one differentiate between Slow and failed?

(2) Reliability :

- In general, distributed systems are expected to be more reliable than centralized systems due to the existence of multiple instances of resources. However, the existence of multiple instances of the resources alone cannot increase the system's reliability.
- Rather, the distributed operating system, which manages these resources, must be designed properly to increase the system's reliability by taking full advantage of this characteristic feature of a distributed system.
- A fault is a mechanical or algorithmic defect that may generate an error. A fault in a system causes system failure. Depending on the manner in which a failed system behaves, system failures are of two types-fail-stop.
- For higher reliability, the fault-handling mechanisms of a distributed operating system must be designed properly to avoid faults, to tolerate faults, and to detect and recover from faults. Commonly used methods for dealing with these issues are fault avoidance and fault tolerance.

(3) Flexibility :

- Another important issue in the design of distributed operating systems is flexibility. Flexibility is the most important feature for open distributed systems.
- The design of a distributed operating system should be flexible due to the following reasons:

- (i) Ease of modification.
- (ii) Ease of enhancement.

(4) Performance :

- If a distributed system is to be used, its performance must be at least as good as a centralized system. That is, when a particular application is run on a distributed system, its overall performance should be better than or at least equal to that of running the same application on a single-processor system.
- However, to achieve this goal, it is important that the various components of the operating system of a distributed system be designed properly; otherwise, the overall performance of the distributed system may turn out to be worse than a centralized system. Some design principles considered useful for better performance are as follows :

(i) Batch if possible.**(ii) Cache whenever possible.****(iii) Minimize copying of data.****(iv) Minimize network traffic.****(v) Take advantage of fine-grain parallelism for multiprocessing.****(5) Scalability :**

- Scalability refers to the capability of a system to adapt to increased service load. It is inevitable that a distributed system will grow with time since it is very common to add new machines or an entire sub network to the system to take care of increased workload or organizational changes in a company.
- Therefore, a distributed operating system should be designed to easily cope with the growth of nodes and users in the system. That is, such growth should not cause serious disruption of service or significant loss of performance to users. Some guiding principles for designing scalable distributed systems are as follows :

(i) Avoid centralized entities.**(ii) Avoid centralized algorithms.****(iii) Perform most operations on client workstations.****(6) Heterogeneity :**

- A heterogeneous distributed system consists of interconnected sets of dissimilar hardware or software systems. Because of the diversity, designing heterogeneous distributed systems is far more difficult than designing homogeneous distributed systems in which each system is based on the same, or closely related, hardware and software.
- However, as a consequence of large scale, heterogeneity is often inevitable in distributed systems. Furthermore, often heterogeneity is preferred by many users because heterogeneous distributed systems provide the flexibility to their users of different computer platforms for different applications.

(7) Security :

- In order that the users can trust the system and rely on it, the various resources of a computer system must be protected against destruction and unauthorized access.

- Enforcing security in a distributed system is more difficult than in a centralized system because of the lack of a single point of control and the use of insecure networks for data communication. Therefore, as compared to a centralized system, enforcement of security in a distributed system has the following additional requirements:
 - It should be possible for the sender of a message to know that the message was received by the intended receiver.
 - It should be possible for the receiver of a message to know that the message was sent by the genuine sender.
 - It should be possible for both the sender and receiver of a message to be guaranteed that the contents of the message were not changed while it was in transfer.
- Cryptography is the only known practical method for dealing with these security aspects of a distributed system.

LIMITATIONS OF DISTRIBUTED SYSTEM

Q.17. Explain limitations of a distributed system.

OR What are inherent limitations of a distribution system? Explain.

Ans. Limitations of a distributed system are as follows :

(I) Absence of a global clock :

- In a distributed system, there does not exist system wide common clock (global clock).
- A reader might think that this problem can be easily solved by either having a clock common to all the computers (processes) in the system or having synchronized clocks, one at each computer.
- Suppose a global (common) clock is available for all the processes in the system.
- In this case, two different processes can observe a global clock at different instants due to unpredictable message transmission delays.
- Therefore, two different processes may falsely perceive two different instants in physical time to be a single instant in physical time.
- On the other hand, if we provide each computer in the system with physical clock and try to synchronize them, these physical clocks can drift from the physical time and the drift rate may vary from clock to clock due to technological limitations.

- Therefore, this approach can also have two different processes running at different computers that perceive two different instants in physical time as a single instant.
 - Hence, we cannot have a system of perfectly synchronized clocks.
- (2) Impact of the absence of global time :
- Due to absence of global time, it is difficult to reason about temporal order of events in a distributed system.
 - Hence, algorithms for distributed system are more difficult to design and debug compared to algorithms for centralized systems.
 - In addition, the absence of a global clock makes it harder to collect up-to-date information on the state of the entire system.
- (3) Absence of shared memory :
- Since the computers in distributed system do not share common memory, an up-to-date state of the entire system is not available to any individual process.
 - Up-to-date state of the system is necessary for reasoning about the system's behaviour, debugging, recovering from failure etc.

LAMPORT'S LOGICAL CLOCKS

Q.18. Explain Lamport's logical clocks in detail. OR

OR Explain happened before relation and causally related events.

OR What is the role of Happened before relation in logical clocks?

Ans. Lamport's logical clocks :

- Lamport proposed the following scheme to order events in a distributed system using logical clocks.
- The execution of processes is characterized by a sequence of events.
- Depending on the application, the execution of a procedure could be one event or the execution of an instruction could be one event.
- When processes exchange messages, sending a message constitutes one event and receiving a message constitutes one event.
- Due to the absence of perfectly synchronized clocks and global time in distributed systems, the order in which two events occur at two different computers cannot be determined based on the local time at which they occur.
- However, under certain conditions, it is possible to ascertain the order in which two events occur based solely on the behaviour exhibited by the underlying computation.

Happened Before Relation ($\rightarrow$):

- The happened before relation captures the causal dependencies between events, i.e. whether two events are causally related or not.
- The relation $\rightarrow$ is defined as follows:
 - $a \rightarrow b$, if a and b are events in the same process and a occurred before b .
 - $a \rightarrow b$, if a is the event of sending a message m in a process and b is the event of receipt of the same message m by another process.
 - If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$, i.e. " $\rightarrow$ " relation is transitive.
- In distributed systems, processes interact with each other and affect the outcome of events of processes.
- Being able to ascertain order between events is very important for designing, debugging and understanding the sequence of execution in distributed computation.
- In general, an event changes the system state, which in turn influences the occurrences and outcome of future events.
- That is, past events influence future events and this influence among causally related events (those events that can be ordered by " $\rightarrow$ ") is referred as causal effects.
- Event ' a ' causally affects event ' b ' if $a \rightarrow b$.

Concurrent events:

- Two distinct events a and b are said to be concurrent (denoted by $a \parallel b$) if $a \not\rightarrow b$ and $b \not\rightarrow a$. In other words, concurrent events do not causally affect each other.
- For any two events a and b in a system, either $a \rightarrow b$, $b \rightarrow a$, or $a \parallel b$.

Example:

- In the space-time diagram of fig, e_{11}, e_{12}, e_{13} and e_{14} are events in process P_1 and e_{21}, e_{22}, e_{23} and e_{24} are events in process P_2 .
- The arrows represent message transfers between the processes. For example, arrow of $e_{12}e_{23}$ corresponds to a message sent from process P_1 to process P_2 , e_{12} is the event of sending the message at P_1 and e_{23} is the event of receiving the same message at P_2 .
- In Fig. we see that $e_{22} \rightarrow e_{13}, e_{13} \rightarrow e_{14}$, and therefore $e_{22} \rightarrow e_{14}$. In other words, event e_{22} causally affects event e_{14} .

Since the occurrence of a is before the occurrence of b , then $a \rightarrow b$. This relation is called the happened before relation.

Events e_{11} and e_{21} are simultaneous events. Events e_{12} and e_{23} are causally related events. Events e_{13} and e_{24} are simultaneous events.

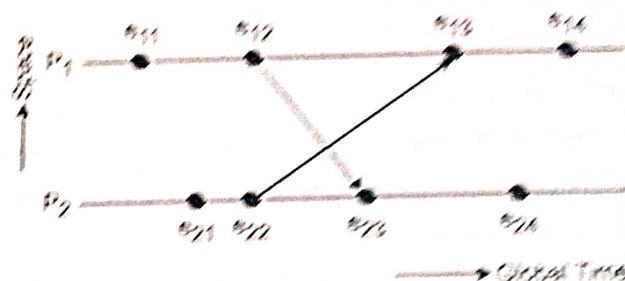


Fig. 1. Space-time diagram

Logical clocks:

- In order to realize the relation $\rightarrow$, Lamport introduced the following system of logical clocks. There is a clock C_i at each process P_i in the system.
- The clock C_i can be thought of as a function that assigns a number $C_i(x)$ to any event x , called the timestamp of event x , at P_i .
- The numbers assigned by the system of clocks have no relation to physical time, and hence the same logical clocks.
- The logical clocks take monotonically increasing values. These clocks can be implemented by counters. Typically, the timestamp of an event is the value of the clock when it occurs.

Q.19. List the conditions satisfied by Lamport's logical clocks.

Ans. Conditions satisfied by the system of clocks:

- For any events a and b :
 - if $a \rightarrow b$, then $C(a) < C(b)$
- The happened before relation " $\rightarrow$ " can now be realized by using the logical clocks if the following two conditions are met:
 - [C1]: For any two events a and b in a process P_i , if a occurs before b , then $C_i(a) < C_i(b)$

- (ii) [C2] : If a is the event of sending a message m in process P_i and b is the event of receiving the same message m at process P_j , then

$$C_i(a) < C_j(b)$$

The following implementation rules (IR) for the clocks guarantee that the clocks satisfy the correctness conditions C1 and C2 :

- i) [IR1] : Clock C_i is incremented between any two successive events in process P_i :

$$C_i := C_i + d \quad (d > 0) \quad \dots\dots(1)$$

If a and b are two successive events in P_i and $a \rightarrow b$, then

$$C_i(b) = C_i(a) + d$$

- ii) [IR2] : If event a is the sending of message m by process P_i , then message m is assigned a timestamp $t_m = C_i(a)$ (note that the value of $C_i(a)$ is obtained after applying rule IR1). On receiving the same message m by process P_j , C_j is set to a value greater than or equal to its present value and greater than t_m .

$$C_j := \max(C_j, t_m + d) \quad (d > 0) \quad \dots\dots(2)$$

Note that the message receipt event at P_j increments C_j as per rule IR1. The updated value of C_j is used in equation (2). Usually, d in equation (1) and (2) has a value of 1.

Lamport's happened before relation, $\rightarrow$, defines an irreflexive partial order among the events.

The set of all the events in a distributed computation can be totally ordered (the ordering relation is denoted by $\Rightarrow$) using the above system of clocks as follows : If a is any event at process P_i and b is any event at process P_j then $a \Rightarrow b$ if and only if either.

$$C_i(a) < C_j(b) \text{ or}$$

$$C_i(a) = C_j(b) \text{ and } P_i < P_j$$

Where, $<$ is any arbitrary relation that totally orders the processes to break ties. A simple way to implement $<$ is to assign unique identification numbers to each process and then $P_i < P_j$, if $i < j$.

Example :

- Fig gives an example of how logical clocks are updated under Lamport's scheme.
- Both the clock values C_{P_1} and C_{P_2} are assumed to be zero initially and d is assumed to be 1. e_{11} is an internal event in process P_1 which causes C_{P_1} to be incremented to 1 due to IR1.
- Similarly, e_{21} and e_{22} are two events in P_2 resulting in $C_{P_2} = 2$ due to IR1.
- e_{16} is a message send event in P_1 which increments C_{P_1} to 6 due to IR1. The message is assigned a timestamp = 6.
- The event e_{25} , corresponding to the receive event of the above message, increments the clock C_{P_2} to 7 (max (4 + 1, 6 + 1)) due to rules IR1 and IR2.
- Similarly, e_{24} is a send event in P_2 . The message is assigned a timestamp = 4.
- The event e_{17} corresponding to the receive event of the above message increments the clock C_{P_1} to 7 (max (6 + 1, 4 + 1)) due to rules IR1 and IR2.

Virtual time :

- Lamport's system of logical clocks implements an approximation to global/physical time, which is referred to as virtual time.
- Virtual time advances along with the progression of events and is therefore discrete.
- If no events occur in the system, virtual time stops, unlike physical time which continuously progresses. Therefore, to wait for a virtual time instant to pass is risky as it may never occur.

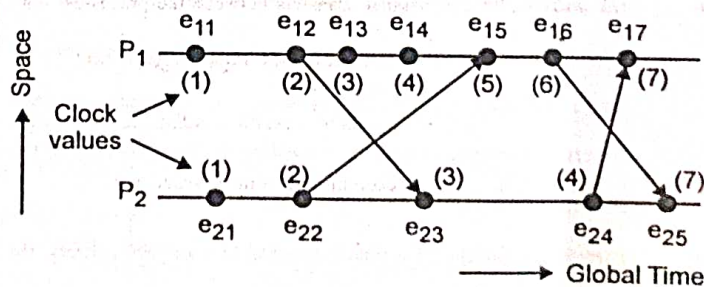


Fig. Lamport's logical clocks advance.

Q.20. Write a short note on limitations of Lamport's clocks.

Ans. The limitation of Lamport's clocks :

- Note that in Lamport's system of logical clocks, if $a \rightarrow b$ then $C(a) < C(b)$. However, the reverse is not necessarily true if the events have occurred in different processes.

- That is, if a and b are events in different processes and $C(a) < C(b)$, then $a \rightarrow b$ is not necessarily true; events a and b may be causally related or may not capture such situations. The example illustrates this limitation of Lamport's clocks.

Example :

- Fig. shows a computation over three processes. Clearly, $C(e_{11}) < C(e_{22})$ and $C(e_{22}) < C(e_{32})$. However, we can see from the figure that event e_{11} is causally related to event e_{22} but not to event e_{32} , since a path exists from e_{11} to e_{22} but not from e_{11} to e_{32} .
- In other words, in Lamport's system of clocks, we can guarantee that if $C(a) < C(b)$ then $b \not\rightarrow a$ (i.e., the future cannot influence the past), however, we cannot say whether events a and b are causally related or not (i.e., whether there exists a path between a and b that moves only forward along the time axis in the space-time diagram) by just looking at the timestamps of the events.

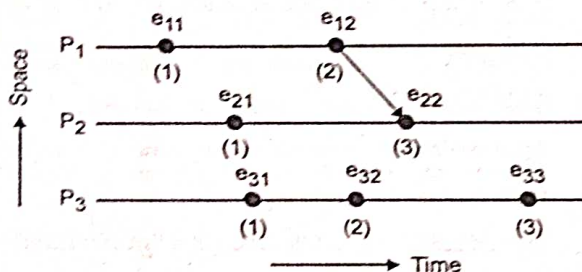


Fig. A space-time diagram

- The reason for the above limitation is that each clock can independently advance due to the occurrence of local events in a process and the Lamport's clock system cannot distinguish between the advancements of clocks due to local events from those due to the exchange of messages between processes.
- Therefore, using the timestamps assigned by Lamport's clocks, we cannot reason about the causal relationship between two events

occurring in different processes by just looking at the timestamps of the events.

VECTOR CLOCKS

Q.21. Explain vector clocks with suitable example.

Ans. Vector clocks :

- The system of vector clocks was independently proposed by Fidge and Mattern.
- A concept similar to vector clocks was proposed previously by Sierum and Yemini for keeping track of transitive dependencies among processes for recovery purposes.

Vector Clock Rules :

- Each process P_i is equipped with a clock C_i which is an integer vector of length n .
- $C_i(a)$ is referred to as the timestamp event a at P_i .
- $C_i[i]$, the i^{th} entry of C_i corresponds to P_i 's on logical time.
- $C_i[j]$, $j \neq i$ is P_i 's best guess of the logical time at P_j .

Implementation rules for vector clocks are as follows :

- [IR1] : Clock C_i is incremented between any two successive events in process P_i .

$$C_i[i] = C_i[i] + d + ab \quad (d > 0)$$
- [IR2] : If event a is the sending of the message m by process P_i , then message m is assigned a vector timestamp $t_m = C_i(a)$; on receiving the same message m by process P_j , C_j is updated as follows :

$$\forall k, C_j[k] := \max(C_j[k], t_m[k])$$

Example :

- Fig. illustrates an example of how clocks advance and the dissemination of time occurs in a system using vector clocks (d is assumed to be 1 and all clock values are initially zero).
- Event e_{11} is an internal event in process P_1 that causes $C_1[1]$ to be incremented to 1 due to IR1.

- e_{12} is a message send event in P_1 which causes $C_1[1]$ to be incremented to 2 due to IR1.
- e_{22} is a message receive event in P_2 that causes $C_2[2]$ to be incremented to 2 due to IR1, and $C_2[1]$ to be set to 2 due to IR2.
- e_{31} is a send event in P_3 which causes $C_3[3]$ to be incremented to 1 due to IR1.
- Event e_{23} , a receive event in P_2 , causes $C_2[2]$ to be incremented to 3 due to IR1, and $C_2[3]$ to be set to 1 due to IR2.
- e_{24} is a send event in P_2 and e_{13} is the corresponding receive event.
- Note that $C_1[3]$ is a set to 1 due to IR2, and process P_1 has learned that the local clock value at P_3 is at least 1 through a message from P_2 .
- Vector timestamps can be compared as follows for any two vector timestamps t^a and t^b of events a and b , respectively :
 Equal : $t^a = t^b$ iff $\forall i, t^a[i] = t^b[i]$;
 Not equal : $t^a \neq t^b$ iff $\exists i, t^a[i] \neq t^b[i]$;
 Less than or equal : $t^a \leq t^b$ iff $\forall i, t^a[i] \leq t^b[i]$;
 Not less than or equal to : $t^a \not\leq t^b$ iff $\exists i, t^a[i] > t^b[i]$;
 Less than : $t^a < t^b$ iff $(t^a \leq t^b \wedge t^a \neq t^b)$;
 Not Less than : $t^a \not< t^b$ iff $(t^a \leq t^b \wedge t^a \neq t^b)$;
 Concurrent : $t^a \parallel t^b$ iff $(t^a \not< t^b \wedge t^b \not< t^a)$;
 Note that the relation " $\leq$ " is a partial order. However, the relation " $\parallel$ " is not a partial order because it is not transitive.

CASUAL ORDERING OF MESSAGES

Q.22. What effect does a communication failure have on a system using the two algorithms for the casual ordering of messages?

Ans. Causal ordering of messages :

- The casual ordering of messages was first proposed by Birman and Joseph.

- The casual ordering of messages deals with the notion of maintaining the same casual relationship that holds among "message send events with the corresponding "message receive" events.
- In other words, if $\text{Send}(M_1) \rightarrow \text{Send}(M_2)$ (where $\text{Send}(M)$ is the event sending message M), then every recipient of both messages M_1 and M_2 must receive M_1 before M_2 .
- The casual ordering of messages should not be confused with the casual ordering of events, which deals with the notion of casual relationship among the events.
- In a distributed system, the casual ordering of messages is not automatically guaranteed.
- For example, Fig shows a violation of casual ordering of messages in a distributed system.
- In this example, $\text{Send}(M_1) \rightarrow \text{Send}(M_2)$ However, M_2 is delivered before M_1 to process P_3 . (In fig, the number in circle indicate the correct casual order to deliver messages)
- Techniques for the casual ordering of messages are useful in developing distributed algorithms and may simplify the algorithms themselves.
- In the absence of causal ordering of messages, each and every update must be checked to ensure that it does not violate the consistency constraints.

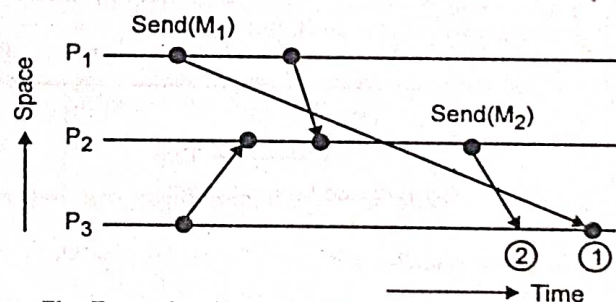


Fig. Example of the violation of casual ordering of message

Q.23. Explain the Birman-Schiper-Stephenson protocol.

Ans. The Birman-Schiper-Stephenson protocol :

- The goal of this protocol is to preserve ordering in the sending of messages.

- For example, if $\text{send}(m1) \rightarrow \text{send}(m2)$, then for all processes that receive both $m1$ and $m2$, $\text{receive}(m1) \rightarrow \text{receive}(m2)$.
- The basic idea is that $m2$ is not given to the process until $m1$ is given. This means a buffer is needed for pending deliveries.
- Also, each message has an associated vector that contains information for the recipient to determine if another message preceded it.
- Also, we shall assume all messages are broadcast. Clocks are updated only when messages are sent.
- The following steps are followed :
- (1) Before broadcasting a message m , a process P_i increments the vector time $VTP_i[i]$ and timestamps m . Note that $(VTP_i[i] - 1)$ indicates how many messages from P_i precede m .
- (2) A process $P_i \neq P_j$, upon receiving message m time stamped VT_m from P_i , delays its delivery until both the following conditions are satisfied.
 - $VTP_j[i] = VT_m[i] - 1$
 - $VTP_j[k] \geq VT_m[k] \forall k \in \{1, 2, \dots, n\} - \{i\}$
 Where, n is the total number of processes.
- Delayed messages are queued at each process in a queue that is sorted by vector time of the messages. Concurrent messages are ordered by the time of their receipt.
- When a message is delivered at a process P_j , VTP_j is updated according to the vector clocks rule [IR2]
- Step 2(a) ensures that process P_j has received the entire message from P_i that precedes m .
- Step 2(b) ensures that process P_j has received the entire message from P_i before sending m .
- Since the event ordering relation " $\rightarrow$ " imposed by vector clocks is acyclic, the protocol is dead lock free.

Q.24. Explain the Schiper-Eggli-Sandoz protocol.

Ans. The Schiper-Eggli-Sandoz protocol :

- The goal of this protocol is to ensure that messages are given to the receiving processes in order of sending.

- Unlike the Birman-Schiper-Stephenson protocol, it does not require using broadcast messages.

- Each message has an associated vector that contains information for the recipient to determine if another message preceded it. Clocks are updated only when messages are sent.

Data Structure and notation :

- Each process P maintains a vector denoted by V_P of size $(N-1)$, where N is a number of processes in the system.
- An element of V_P is an order pair (P', t) where P' is ID of the destination process of a message and t is a vector timestamp.
- The processes in the system are assumed to use vector clocks.
- The communication channels can be non-FIFO.
- The following notations are used in describing the protocol:

(i) t_M = logical time at the sending of message M .

(ii) t_{P_i} = present/current logical time at process P_i .

Sending of a message M from process P_1 to process P_2 :

- Send message M (timestamped t_M) along with V_P_1 to process P_2 .
- Insert pair (P_2, t_M) into V_P_1 . If V_P_1 contains a pair (P_2, t) it simply gets over written by the new pair (P_2, t_M) .
- Note that the pair (P_2, t_M) was not sent to P_2 . Any future message carrying the pair (P_2, t_M) cannot be delivered to P_2 until $t_M < t_{P_2}$.

Arrival of a message M at process P_2 :

- If V_M (the vector accompanying message M) does not contain and any pair (P_2, t) then the message can be delivered
- else (* A pair (P_2, t) exists in V_M *)
- If $t < t_{P_2}$ then,
- the message cannot be delivered (* it is buffered for later delivery *)
- else
- the message can be delivered.

- If message M can be delivered at process P_2 , then the following three actions are taken :
 - Merge V_M accompanying M with V_{P_2} in the following manner :
 - If $(\exists (P, t) \in V_M, \text{ such that } P \neq P_2) \text{ and } (\forall (P', t') \in V_{P_2}, P' \neq P)$, then insert (P, t) into V_{P_2} . This rule performs the following : if there is no entry for process P in V_{P_2} and V_M contains an entry for process P , insert that entry into V_{P_2} .
 - $\forall P, P \neq P_2$ if $((P, t) \in V_M) \wedge ((P, t') \in V_{P_2})$, then the algorithm takes the following actions : $(P, t) \in V_{P_2}$ can be substituted by the pair (P, t_{sup}) where t_{sup} is such that $\forall i, t_{sup}[i] = \max(t[i], t'[i])$.

Due to the above two actions, the algorithm satisfies the following two conditions :

- No message can be delivered to P as long as $t' < t_p$ is not true.
 - No message can be delivered to P as long as $t < t_p$ is not true.
- Update site P_2 's logical clock.
 - Check for the buffered messages that can now be delivered since local clock has been updated.

A pair (P, t) can be deleted from the vector maintained at a site after ensuring that the pair (P, t) has become obsolete (i.e. no longer needed).

GLOBAL STATE RECORDING

Q.25. What do you understand by local state and global state?

Ans. Local state :

- For a site (computer) S_i , its local state at a given time is defined by the local context of the distributed application.
- Let LS_i denote the local state of S_i at any time.
- Let $send(m_{ij})$ denote the send event of a message m_{ij} by S_i to S_j , and $rec(m_{ij})$ denote the receive event of message m_{ij} by site S_j .
- Let $time(x)$ denote the time at which state x was recorded and $time(send(m))$ denote the time at which event $send(m)$ occurred.

- For a message m_{ij} sent by S_i to S_j , we say that
 - $send(m_{ij}) \in LS_i$ iff $time(send(m_{ij})) < time(LS_i)$
 - $rec(m_{ij}) \in LS_j$ iff $time(rec(m_{ij})) < time(LS_j)$
- For the local states LS_i and LS_j of any two sites S_i and S_j , we define two sets of messages. These sets contain messages that were sent from site S_i to site S_j .

(i) **Transit :**

$$transit(LS_i, LS_j) = \{m_{ij} \mid send(m_{ij}) \in LS_i \wedge rec(m_{ij}) \notin LS_j\}$$

(ii) **Inconsistent :**

$$inconsistent(LS_i, LS_j) = \{m_{ij} \mid send(m_{ij}) \notin LS_i \wedge rec(m_{ij}) \in LS_j\}$$

Global state : A global state, GS , of a system is a collection of the local states of its sites : That is, $GS = \{LS_1, LS_2, \dots, LS_n\}$ where, n is the number of sites in the system.

(i) **Consistent global state :**

- A global state $GS = \{LS_1, LS_2, \dots, LS_n\}$ is consistent iff

$$\forall i, \forall j: 1 \leq i, j \leq n :: inconsistent(LS_i, LS_j) = \phi$$

Thus, in a consistent global state, for every received message a corresponding send event is recorded in the global state.

- In an inconsistent global state, there is at least one message whose receive event is recorded but its send event is not recorded in the correspond to consistent and inconsistent global states, respectively.

(ii) **Transitless global state :**

A global state is transitless if and only if

$$\forall i, \forall j: 1 \leq i, j \leq n :: transit(LS_i, LS_j) = \phi$$

Thus, all communication channels are empty in a transitless global state.

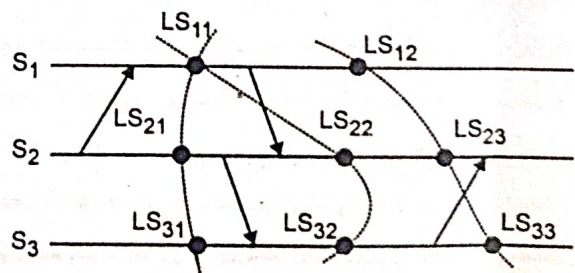


Fig. Global states in a distributed computation.

(iii) Strongly consistent global state :

- A global state is strongly consistent if it is consistent and transitless.
- In a strongly consistent state, not only the send events of all the recorded received events are recorded, but the receive events of all the recorded send events are also recorded. Thus, a strongly consistent state corresponds to a consistent global state in which all channels are empty.
- In fig. the global state $\{LS_{11}, LS_{21}, LS_{31}\}$ is a strongly consistent global state.

Q.26. Explain with a suitable example how will you find out that global state is consistent or not.

Ans.

- A global state, GS, of a system is a collection of the local states of its site; That is, $GS = \{LS_1, LS_2, LS_3, \dots, LS_n\}$ where, n is the number of sites in the system.
- Note that any collection of local states of sites needs not represent a consistent global state.
- Consistency has a connotation that for every effect or outcome recorded in global state, the cause of the effect must also be recorded in the global state.
- We now address the problem of collecting or recording a coherent (consistent) global state in distributed system, a challenging task due to the absence of global clock and shared memory.

Banking example :

- Fig. shows the stages of computation when \$50 is transferred from account A to account B. The communication channels C1 and C2 are assumed to be FIFO.
- Suppose the state of account A at site S1 was recorded when the global state was 1.
- Now, assume that the global state changes to 2, and the state of communication channel C1 and C2 and account B are recorded when global state is 2.
- Then composite of all the states recorded would show account A's balance as \$500, account B's balance as \$200, and a message in transit to transfer \$50.

- In other words, an extra amount of \$50 would appear in the global state.
- The reason for this inconsistency is that A's state was recorded before the message was sent and the channel C1's state was recorded after the message was sent.
- Therefore, a recorded global state may be inconsistent if $n < n'$ where, n is number of message sent by A along the channel before A's state was recorded and n' is the number of messages sent by A along the channel before the channel's state was recorded.

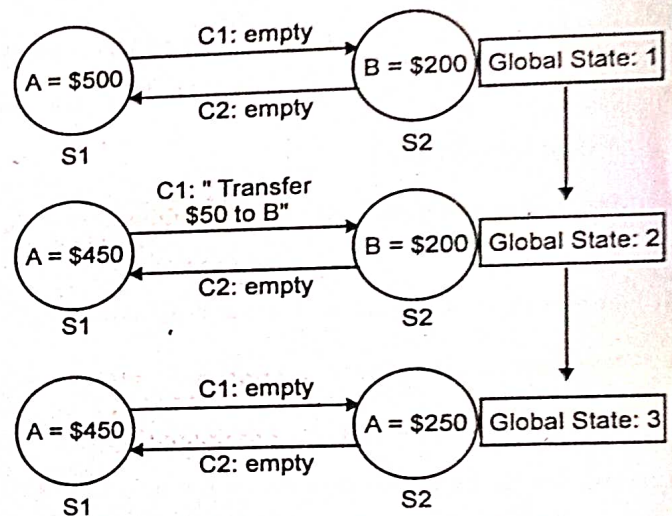


Fig. Global state and their transitions in the bank account example

Q.27. Explain Chandy-Lamport's algorithm for state recording.

Ans. Chandy-Lamport's Global state recording algorithm :

- Chandy and Lamport were the first to propose a distributed algorithm to capture a consistent global state.
- The algorithm uses a marker (a special message) to initiate the algorithm and the marker has no effect on the underlying computation.
- The communication channels are assumed to be FIFO.
- The recorded global state is also referred to as a snapshot of the system state.

Marker sending rule for a process P :

- P records its state.
- For each outgoing channel C from P on which a marker has not been already sent, P sends a marker along C before P sends further messages along C.

Marker receiving rule for a process Q :

On receipt of a marker along a channel C :

If Q has not recorded its state

then

begin

Record the state of C as an empty sequence.

Follow the "Marker Sending Rule".

end

else

Record the state of C as the sequence of messages received along C after Q's state was recorded and before Q received the marker along C.

- The role of markers in conjunction with FIFO channels is to act as delimiters for the messages in the channels so that the channel state recorded by the process at the receiving end of the channel satisfies the condition $(n \geq m)$.
- A marker delineates messages into those that need to be included in the recorded state and those that are not to be recorded in the state.
- The global state recording algorithm can be initiated by any process by executing the marker sending rule. Also, the global state recording algorithm can be initiated by several processes concurrently with each process getting its own version of a consistent global state.
- Each initiation needs its own unique marker (such as <process-id, sequence number>) and different initiations by a process can be distinguished by a local sequence number.
- A simple way to collect all the recorded information is for each process to send the information it recorded to the initiator of the recording process. The identification of the initiator process can be easily carried by the marker.

Q.28. Write a short note on the collected global state.

Ans. Collected global state :

- It is possible that the global state recorded by the Chandy-Lamport's algorithm is not identical to any of the global states the system actually went through during the computation. This can happen

because a site can change its state asynchronously before the markers sent by it are received by other sites.

- If the state changes while the markers are in transit, the composite of all the states recorded will not correspond to the state of the system at any instant of time.
- The question of the significance of the recorded global state if the system may have never passed through it arises.
- Suppose the algorithm is initiated when the system is in global state S_i and it terminates when the system is in global state S_t .
- Let S_c denote the collected global state by the algorithm, and Seq' denote the sequence of actions which take the system from state S_i to S_t . Then, there exists a sequence Seq' that is a permutation of Seq such that S_c can be reached from S_i by executing a prefix of Seq' and S_t can be reached from S_c by executing the rest of the actions in Seq' (see fig.).
- The usefulness of the recorded global state lies in its ability to detect stable properties (a stable property is one that persists) such as the termination of a computation and a deadlock among processes.
- Note that if a stable property holds before the recording algorithm begins execution, it continues to hold (unless resolved in the case of a deadlock) and will therefore be included in the recorded global state.
- Even though the global state recording algorithm can be used for termination detection, it is an expensive way of doing it.

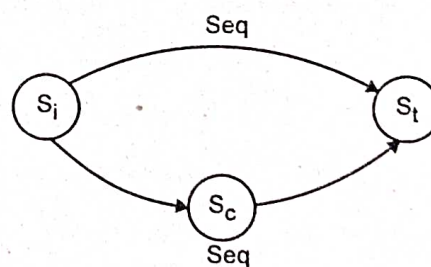


Fig. A property of a collected global state

CUTS OF DISTRIBUTED COMPUTATION

Q.29. Explain in detail the cuts in a distributed computation.

Ans. Cut :

- The notion of a cut captures a global state in a distributed computation.

- A cut is a graphical representation of a global state in the history of distributed computation.
- A cut of a distributed computation is a set $C = \{c_1, c_2, \dots, c_n\}$, where c_i is the cut event at site S_i in the history of distributed computation.
- Graphically, a cut is a zig-zag line that connects the corresponding cut events in the time-space diagram. For example, in fig (a) events c_1, c_2, c_3 and c_4 form a cut.
- If a cut event c_i at site S_i is S_i 's local state at that instant, then clearly a cut denotes a global state of the system.

- A consistent cut is a graphical representation of a consistent global state in the history of a distributed computation.

Consistent cut :

- A consistent cut is a graphical representation of a consistent global state in the history of a distributed computation.
- Let c_k denote an event at site S_k . A cut $C = \{c_1, c_2, \dots, c_n\}$ is a consistent cut iff

$$\forall S_i, \forall S_j, \exists e_i, \exists e_j \text{ such that } (e_i \rightarrow e_j) \wedge (e_j \rightarrow c_i) \wedge (e_i \not\rightarrow c_i)$$

where, $c_i \in C$ and $c_j \in C$.

- That is, a cut is a consistent cut if every message that was received before a cut event was sent before the cut event at the sender site in the cut. For example, the cut in fig. (a) is not consistent because the message sent by S_2 is received before c_3 but the corresponding send did not occur before event c_2 . That is, $c \rightarrow e', e' \rightarrow c_3$, and $c \not\rightarrow c_2$.

Theorem :

- A cut $C = \{c_1, c_2, \dots, c_n\}$, is a consistent cut if and only if no two cut events are causally related, that is, $\forall c_i, \forall c_j :: \neg(c_i \rightarrow c_j) \wedge \neg(c_j \rightarrow c_i)$
- Consider two events c_1 and c_2 in fig. (b) which are not concurrent because $c_1 \rightarrow c_2$.
- Clearly, c_1 and c_2 do not lie on a consistent cut.
- On the other hand, if two events are concurrent, it implies that no message sent after one of them has been received before the other one. Thus, concurrent cut events form a consistent cut.

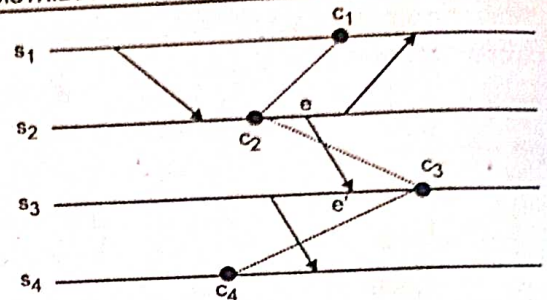


Fig.(a) A cut

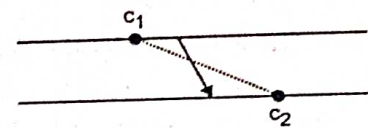


Fig.(b) A cut where cut are not concurrent

Time of a cut :

- If each cut event is assigned a vector timestamp, then a cut in a distributed computation can be assigned a timestamp in the following manner :

- If $C = \{c_1, c_2, \dots, c_n\}$ is a cut, where c_i is a cut event at site S_i with vector timestamp VT_{c_i} , then the vector time of the cut, VT_c is defined as follows :

$$VT_c = \sup(VT_{c_1}, VT_{c_2}, \dots, VT_{c_n})$$

Where, sup is a componentwise maximum operation. That is,

$$VT_c[i] = \max(VT_{c_1}[i], VT_{c_2}[i], \dots, VT_{c_n}[i])$$

Theorem : If $C = \{c_1, c_2, \dots, c_n\}$ is a cut with a vector time VT_c , then the cut is consistent iff

$$VT_c = (VT_{c_1}[1], VT_{c_2}[2], \dots, VT_{c_n}[n])$$

Proof :

- If C is a consistent cut, then from the previous theorem, events $c_1, c_2, \dots, c_n$ are concurrent. In other words, $\forall i, \forall j, c_i[i] \geq c_j[i]$. Therefore,
- $$\sup(VT_{c_1}, VT_{c_2}, \dots, VT_{c_n}) = (VT_{c_1}[1], VT_{c_2}[2], \dots, VT_{c_n}[n])$$
- For sufficiency, note that $VT_c = (VT_{c_1}[1], VT_{c_2}[2], \dots, VT_{c_n}[n])$ implies that no message sent after a cut event c_i has been received before another cut event c_j . Thus, cut events $c_1, c_2, \dots, c_n$ form a consistent cut.

TERMINATION DETECTION

Q.30. Briefly explain termination detection algorithm in distributed operating system.

OR Write short note on Huang's termination detection algorithm.

Ans. Termination detection :

- A distributed computation generally consists of a set of cooperating processes which communicate with each other by exchanging messages.
- In case of distributed computation, it is important to know when the computation has terminated.
- The problem of termination detection arises in many distributed algorithms and computations.
- For example, how to determine when an election, a deadlock detection, a deadlock resolution, or a token generating algorithm has terminated.
- Termination detection is an example of the usage of the coherent view (consistent global state) of a distributed system.
- A large number of algorithms have been developed for termination detection, we are going to discuss termination detection algorithm proposed by Huang.

Huang's termination detection algorithm :

- The goal of this protocol is to detect when a distributed computation terminates.
- The following notations are used in the algorithm :
 - (i) n processes.
 - (ii) P_i process; without loss of generality, let P_0 be the controlling agent.
 - (iii) W_i weight of process P_i ; initially, $W_0 = 1$ and for all other i , $W_i = 0$.
 - (iv) $B(W)$ computation message with assigned weight W .
 - (v) $C(W)$ control message sent from process to controlling agent with assigned weight W .

Algorithm :

Rule 1 : P_i sends a computation message to P_j :

- (i) Set W_i' and W_j to values such that $W_i' + W_j = W_i$, $W_i' > 0$, $W_j > 0$. (W_i' is the new weight of P_i).
- (ii) Send $B(W_j)$ to P_j .

Rule 2 : P_j receives a computation message $B(W)$ from P_i :

- (i) $W_i = W_i + W$
- (ii) If P_j is idle, P_j becomes active.

Rule 3 : P_j becomes idle :

- (i) Send $C(W_i)$ to P_0 .
- (ii) $W_i = 0$.
- (iii) P_j becomes idle.

Rule 4 : P_0 receives a control message $C(W)$:

- (i) $W_0 = W_0 + W$.
- (ii) If $W_0 = 1$, the computation has completed.

Example :

- The picture shows a process P_0 , designated the controlling agent, with $W_0 = 1$.
- It asks P_1 and P_2 to do some computation. It sets W_1 to 0.2, W_2 to 0.3, and W_3 to 0.5. P_2 in turn asks P_3 and P_4 to do some computations. It sets W_3 to 0.1 and W_4 to 0.1.
- When P_3 terminates, it sends $C(W_3) = C(0.1)$ to P_2 , which changes W_2 to $0.1 + 0.1 = 0.2$.
- When P_2 terminates, it sends $C(W_2) = C(0.2)$ to P_0 , which changes W_0 to $0.5 + 0.1 = 0.7$.
- When P_4 terminates, it sends $C(W_4) = C(0.1)$ to P_0 , which changes W_0 to $0.7 + 0.1 = 0.8$.
- When P_1 terminates, it sends $C(W_1) = C(0.2)$ to P_0 , which changes W_0 to $0.8 + 0.2 = 1$.
- P_0 there upon concludes that the computation is finished.
- Total number of messages passed: 8 (one to start each computation, one to return the weight).

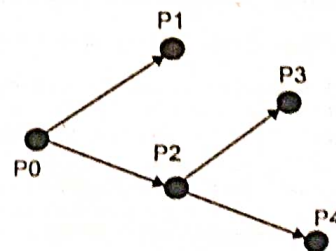


Fig. Example of termination detection